

CA4022 – Data (Analytics) at Speed and Scale

Alessandra Mileo

alessandra.mileo@dcu.ie

Our Spark journey

- Part 1: Spark Introduction
- Part 2: RDDs
- Part 3: Using Spark
- **Part 4: Machine Learning with Spark**
 - ML tasks in Spark: brief intro
 - DataFrames and DataSets in the context of ML

Spark data collection types

- RDDs (from start until now): easy to use (similarly to Java 8 streams), stable and flexible - but not most efficient
- **DataFrame** (since Spark 1.3): fast, stable, close to SQL, restricted expressiveness compared to RDDs
- DataSets (since Spark 1.6 as a preview, standard data structure since 2.0): kind of mix between RDDs and DataFrames, fast

Spark DataFrames

- Distributed, Immutable
- *row-oriented* (i.e., each "data record" is a row in a table)
- Facilitated by Spark SQL
- DataFrames can be constructed from, e.g.,
 - Data files (e.g., ordinary text files, JSON documents...)
 - RDBMS tables
 - Apache Hive tables
 - RDDs
 - Java collections (e.g., lists)

DataFrame vs RDD

- Main features over RDDs:
 - More restricted, therefore easier to optimize for the framework
 - They carry more meta-information (schema)
- Main shortcomings:
 - More restricted, therefore more difficult to apply in some cases
 - Requires manual schema attachment or deduction of schema
 - Weakly typed
 - Replaced by DataSets since 2.0 for Java & Scala (not for Python and R)

Note: DataFrame = DataSet<Row>

DataFrames: query and ML

- SQL-like query:

Obtains so-called SQLContext
from SparkContext



```
SQLContext sqlContext = new org.apache.spark.sql.SQLContext(sc)
DataFrame df =
sqlContext.read().json("examples/src/main/resources/people.json");
df.filter(df.col("age").gt(21)).show(); // Select all people older than 21
```

Real SQL:

```
DataFrame cheapB = sqlContext.sql("SELECT book FROM books WHERE price  
<= 10");
```

- Machine Learning differences (e.g. ML pipelines)

DataFrame example

`people.json`

```
{ "name": "Michael" }  
{ "name": "Andy", "age": 30 }  
{ "name": "Justin", "age": 19 }
```

```
df = sqlContext.read.json("people.json")  
df.show()
```

```
+----+-----+  
| age|   name|  
+----+-----+  
| null|Michael|  
|  30|   Andy|  
|  19|  Justin|  
+----+-----+
```

```
df.printSchema()
```

```
root  
|-- age: long (nullable = true)  
|-- name: string (nullable = true)
```

i.e., DataFrames have a schema (just like database tables)

From RDDs to DataFrames

- In Java:

```
SQLContext sqlContext = new org.apache.spark.sql.SQLContext(sc);  
JavaRDD<Person> books = sc.textFile("books.txt").map(...);  
// Apply a schema to an RDD of books and register it as a table.  
// Important: class Book needs to be a JavaBean class  
DataFrame schematedBooks = sqlContext.createDataFrame(books, Book.class);  
schematedBooks.registerTempTable("books");
```

- In general, the way to do it is using

```
sc.createDataFrame(RDD, schema)
```


From RDDs to DataFrames

- In Python (without specifying the schema)

```
booksRDD = sc.textFile("books.txt").map(...) //Create RDD
booksDF = booksRDD.toDF()                    //Convert to DataFrame
booksDF.printSchema()                       //print schema in a tree format
booksDF.show(truncate=False)                //show results (no truncation)
```

- In Python (specifying the schema)

```
...
booksDF = booksRDD.toDF("author","title","year")
//Convert to DataFrame with column names
...
```

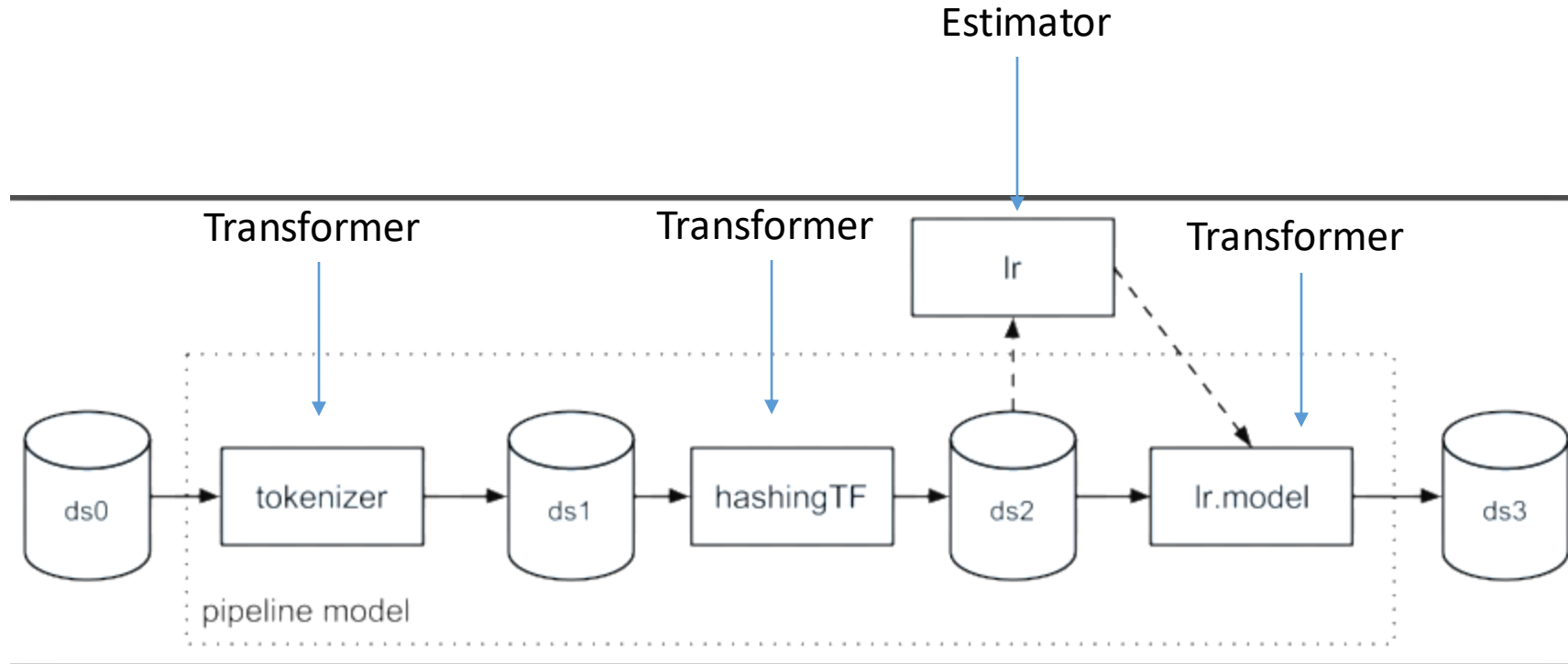
ML Pipelines basics

- On DataFrames (and DataSets)
- Set of API (spark.ml)
- Optional to specify ML workflows
- Reflect ML steps
 - Prepare data
 - Extract features
 - Learn model
 - Evaluate model
 - Predict

ML Pipelines basics

- Sequence of stages, transforming and fitting data frames
- Each stage is either a *Transformer* or an *Estimator*
- For Transformer stages, the transform() method is called on the data frame to produce a model or to test it
- For Estimator stages, the fit() method is called on a DataFrame to create a model (train it)

Machine Learning with Spark



Source: DataBricks

Examples (for next week's lab)

Python Version (next week's lab):

- Spam/Ham email classification

Do it yourself (breakout rooms):

- *Sentiment Analysis (good/bad)*

Do it yourself (breakout rooms):

- *Modify to add neutral or*
- *Modify to use SVM instead*

DataSets and DataFrames

- DataFrame = Dataset<Row>
- Dataset<Book> instead of Dataset<Row> (a.k.a. DataFrame)
- Faster encoders
- Same properties as DataFrames (relationship with DB tables)

DataSets example (JAVA)

- Creating a DataSet of book objects
- Would work with DataFrames too, but "<Book>" would become the unspecific "<Row>" (and field names would be mere strings)
- Observe that class Book needs to 1) implement interface Serializable and 2) be a JavaBean (i.e., have public getter and setter methods)

```
class Book implements Serializable {  
  
    String title;  
    int year;  
  
    public String getTitle() { // we need getters/setters, i.e., Book needs to be a JavaBean class  
        return title;  
    }  
    public void setTitle(String title) {  
        this.title = title;  
    }  
    public int getYear() {  
        return year;  
    }  
    public void setYear(int year) {  
        this.year = year;  
    }  
}
```


Encoders

- Obtaining DataSets from, e.g., DataFrames or RDDs using encoders:

```
Dataset<Double> dataSetOfDoubles = dataSetOfRows.map( row -> {  
    Double d = row.getAs("fieldname");  
    return d;  
}, Encoders.DOUBLE() );
```

now typed!

DataFrame (of rows
with untyped/stringly
typed columns (fields))

```
Book someBook = new Book();  
someBook.setTitle("1001 Recipes for Fish");  
someBook.setYear(1999);
```

Encoder<Book> personEncoder = Encoders.bean(Book.class); // (there are also encoders for standard data types like numbers, tuples and arrays, see further example later)

```
Dataset<Book> javaBeanDS = sqlContext.createDataset(  
    Collections.singletonList(someBook),  
    personEncoder  
);
```

```
javaBeanDS.show();
```

Java compile-time
checked field id



```
Dataset<Book> filteredBooks = javaBeanDS.filter(b -> b.year > 30);
```

Datasets Shortcomings

- Duplicated class names (spark.ml and spark.mllib)
- Mix of DataSets, DataFrames and RDDs
- Availability (Java/Scala)
- Need to rewrite code for ML on RDDs

End of ML with spark.ml

DataFrames and DataSets